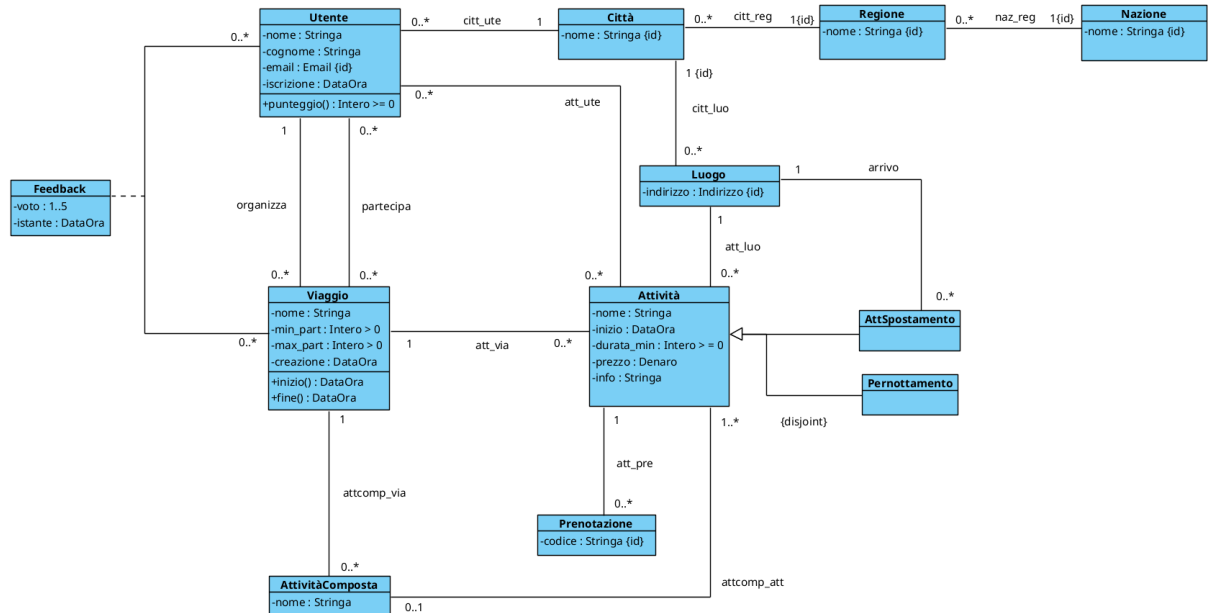


TRAVELPLAN

DIAGRAMMA DELLE CLASSI



SPECIFICA DEI TIPI DI DATO

- Email

Segue standard email

- Denaro

(Valore: Reale >= 0, Valuta: Segue standard (EUR, USD, ecc...))

- Indirizzo

(Via: Stringa, civico: Intero > 0, CAP: [0-9]{5})

SPECIFICA DELLE CLASSI

- Classe Attività

[V.Attività.no_utente_se_non_partecipa_al_viaggio]

```
!EXISTS ute, via |  
    att_ute(this, ute) AND  
    att_via(this, via) AND  
    !partecipa(ute, via)
```

[V.Attività.no_prima_creazione_viaggio]

```
!EXISTS via, crea, ini |  
    att_via(this, via) AND  
    creazione(via, crea) AND  
    inizio(this, ini) AND  
    ini < crea
```

- Classe AttivitàComposta

[V.AttivitàComposta.tutte_attività_stesso_viaggio]

```
FORALL att |  
    attcomp_att(this, att) ->  
        EXISTS via |  
            attcompvia(this, via) AND  
            att_via(att, via)
```

- Classe Utente

[V.Utente.max_un_pernottamento_al_giorno]

FORALL via | partecipa(this, via) ->

!EXISTS p1, p2, ini1, ini2, d |

Pernottamento(p1) AND att_via(p1, via) AND
Pernottamento(p2) AND att_via(p2, via) AND
p1 != p2 AND

inizio(p1, ini1) AND data(ini1, d) AND
inizio(p2, ini2) AND data(ini2, d) AND

(att_ute(p1, this) OR (!EXISTS u1 | att_ute(p1, u1))) AND

(att_ute(p2, this) OR (!EXISTS u2 | att_ute(p2, u2)))

punteggio(): Intero >= 0

pre:

EXISTS u', via |
organizza(this, via) AND
feedback(u', via)

post:

result soddisfa la seguente formula:

FEEDS = { (u', via, v) |
organizza(this, via) AND
feedback(u', via) AND
voto(u', via, v) }

```

AND
media = ( SUM( v | EXISTS u, via |
              (u, via, v) in FEEDS) / |FEEDS| )

AND
media <= 3 -> result = 0

AND
media > 3 ->
    result = PARTEINTERAINF( 0.10 * |{ u, via |
                              EXISTS v |
                                (u, via, v) in FEEDS AND
                                v >= 4 }| )

```

- Classe Feedback

[V.Feedback.no_prima_iscrizione_utente]

```

!EXISTS ist, ute, iscr, via |
    feedback(ute, via) AND
    istante(ute, via, ist) AND
    iscrizione(ute, iscr) AND
    ist < iscr

```

[V.Feedback.dopo_fine_viaggio]

```

!EXISTS ute, via, ist, fin |
    feedback(ute, via) AND
    istante(ute, via, ist) AND
    fine(via, fin) AND
    ist < fin

```

[V.Feedback.solo_se_utente_partecipa]

```
!EXISTS ute, via |  
    feedback(ute, via) AND  
    !partecipa(ute, via)
```

- Classe Viaggio

[V.Viaggio.no_creato_prima_iscrizione_utente]

```
!EXISTS ute, iscr, crea |  
    organizza(ute, this) AND  
    creazione(this, crea) AND  
    iscrizione(ute, iscr) AND  
    crea < iscr
```

inizio(): DataOra

```
pre:  
    EXISTS att |  
        att_via(att, this)  
post:  
    result soddisfa la seguente formula:  
  
    result = min( ini | EXISTS att |  
                                att_via(att, this) AND  
                                inizio(att, ini) )
```

fine(): DataOra

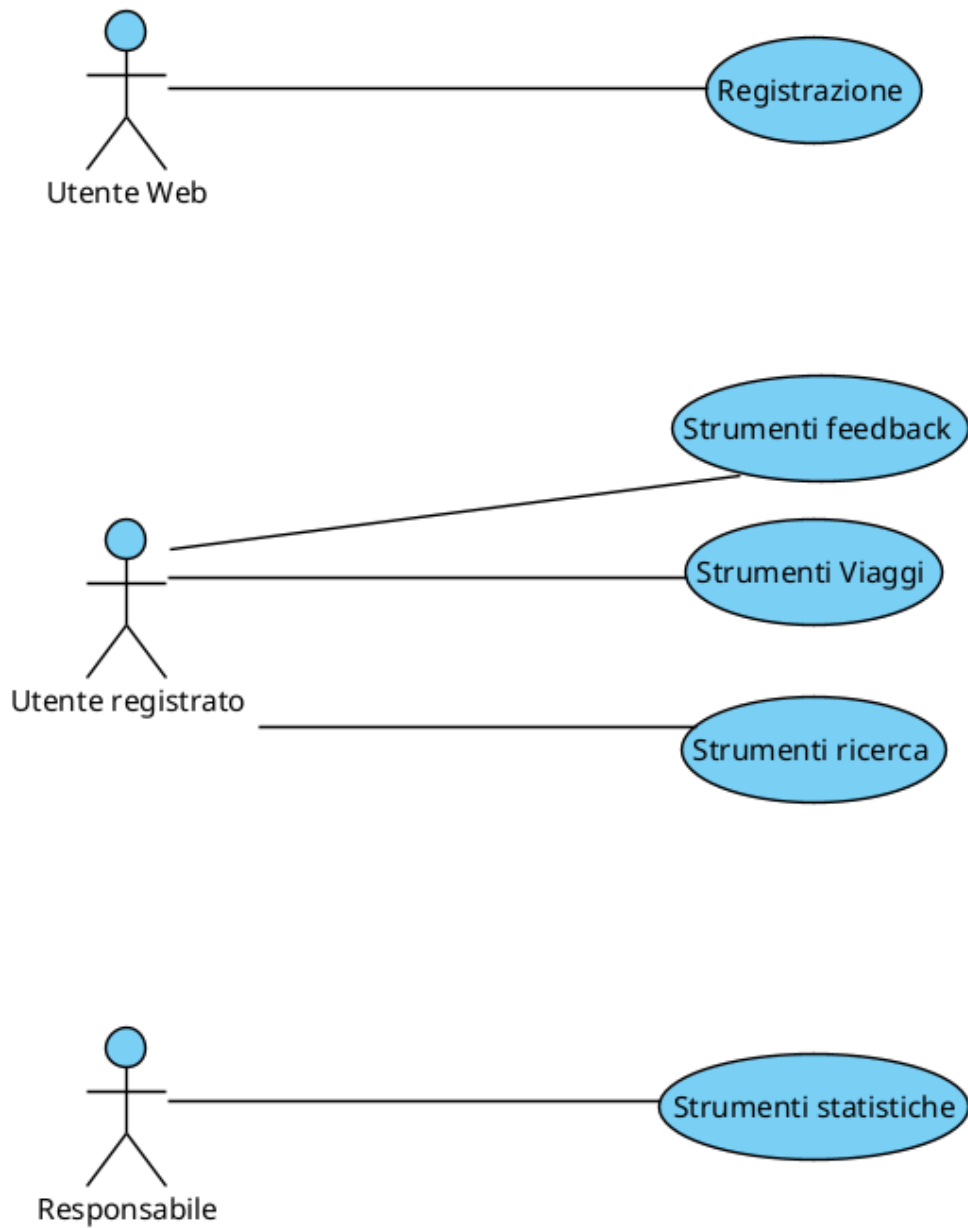
```
pre:  
    EXISTS att |  
        att_via(att, this)
```

post:

result soddisfa la seguente formula:

```
result = max ( r | EXISTS a, ini, dur |  
                att_via(a, this) AND  
                inizio(a, ini) AND  
                durata_min(a, dur) AND  
                r = ini + (dur * "1 minuto") )
```

DIAGRAMMA DEGLI USE-CASE



SPECIFICA DEGLI USE-CASE

- Use-Case Registrazione

**nuovo_utente(n:String, c:String, e:Email,
 citt:Città): Utente**

pre:

!EXISTS u' |
 email(u', e)

post:

il livello estensionale finale differisce
da quello iniziale come segue:

nuovi elementi del dominio: alpha

elementi rimossi dal dominio: nessuno

nuove ennuple:

- Utente(alpha)
- nome(alpha, n)
- cognome(alpha, c)
- email(alpha, e)
- citt_ute(citt, alpha)

ennuple rimosse: nessuna

valore di ritorno: result = alpha

- Use-Case Strumenti viaggi

**crea_viaggio(n:String, minp:Inter>0,
maxp:Inter>0): Viaggio**

pre:

nessuna

post:

il livello estensionale finale differisce
da quello iniziale come segue:

nuovi elementi del dominio: alpha

elementi rimossi dal dominio: nessuno

nuove ennuple:

- Viaggio(alpha)
- nome(alpha, n)
- min_part(alpha, minp)
- max_part(alpha, maxp)
- creazione(alpha, adesso)

sia u:Utente l'istanza dell'utente che ha
invocato lo use-case

- organizza(u, alpha)

ennuple rimosse: nessuna

valore di ritorno: result = alpha

**aggiungi_attività_semplice(v:Viaggio,
n:Stringa, ini:DataOra, dur:Inter0=>0, pre:Denaro,
inspren:Prenotazione[0..*], luo:Luogo,**

**tipo:{semplice, spostamento,
pernott},luo_fine:Luogo[0..1], inf:Stringa**

): Attività

pre:

sia u:Utente l'istanza dell'utente che ha invocato
lo use-case

organizza(u, v)

AND

tipo = spostamento <-> luo_fine != null

AND

EXISTS crea |

creazione(v, crea) AND

crea < ini

post:

il livello estensionale finale differisce
da quello iniziale come segue:

nuovi elementi del dominio: alpha

elementi rimossi dal dominio: nessuno

nuove ennuple:

- Attività(alpha)

- tipo = spostamento -> Spostamento(alpha)

arrivo(alpha, luo_fine)

- tipo = pernottamento -> Pernottamento(alpha)

- nome(alpha, n)

- att_via(alpha, v)

- att_luo(alpha, luo)
- inizio(alpha, ini)
- durata_min(alpha, dur)
- prezzo(alpha, pre)
- info(alpha, inf)
- att_pre(alpha, pre) per ogni pre in inspren

ennuple rimosse: nessuna

valore di ritorno: result = alpha

**aggiungi_attività_composta(n:String, v:Viaggio,
insa:Attività[1..*]): AttivitàComposta**

pre:

sia u:Utente l'istanza dell'attore che ha invocato
lo use-case

FORALL att |
 att in insa -> att_via(att, v)
AND
organizza(u, v)

post:

il livello estensionale finale differisce
da quello iniziale come segue:

nuovi elementi del dominio: alpha
elementi rimossi dal dominio: nessuno
nuove ennuple:

- AttivitàComposta(alpha)
- nome(alpha, n)
- attcomp_via(alpha, v)
- attcomp_att(alpha, att) per ogni att in insa

ennuple rimosse: nessuna
valore di ritorno: result = alpha

- Use-Case Strumenti ricerca

**trova_viaggi_dest(luo:Luogo, ini:DataOra,
fin:DataOra): Viaggio[0..*]**

pre:

nessuna

post:

questo use-case non modifica il livello estensionale
result soddisfa la seguente formula:

```
result = { via | EXISTS att, iniv, finv |  
                                att_via(att, via) AND  
                                inizio(via, iniv) AND  
                                fine(via, finv) AND  
                                iniv >= ini AND  
                                finv <= fin AND  
                                ( att_luo(att, luo)  
                                OR arrivo(att, luo) ) }
```

**trova_città_popolari(ini:DataOra, fin:DataOra):
Città[0..*]**

pre:

nessuna

post:

questo use-case non modifica il livello estensionale
result soddisfa la seguente formula:

```

POPOLARITÀ = { (c, n) | Città(c) AND
                n = |{ via | EXISTS att, luo, iniv, finv |
                    att_via(att, via) AND
                    inizio(via, iniv) AND
                    fine(via, finv) AND
                    iniv >= ini AND
                    finv <= fin AND
                    ( att_luo(att, luo)
                      OR arrivo(att, luo) ) AND
                    citt_luo(c, luo) AND}| }

AND

maxn = max( n | EXISTS c | (c, n) in POPOLARITÀ )

AND

result = { c | EXISTS num |
           (c, num) in POPOLARITÀ AND
           num = maxn }

```

calcola_viaggi_nazione(naz:Nazione, ini:DataOra,
fin:DataOra): (r:Regione, nv:Inter0>=0)

pre:

nessuna

post:

questo use-case non modifica il livello estensionale
 result soddisfa la seguente formula:

```
REGIONI = { r | naz_reg(naz, r) }
```

AND

```
result = { (r, nvis) | r in REGIONI AND
```

```
nvis = |{ via | EXISTS att, luo, c, iniv, finv |
        att_via(att, via) AND
```

```

        inizio(via, iniv) AND
        fine(via, finv) AND
        iniv >= ini AND
        finv <= fin AND
        ( att_luo(att, luo) OR arrivo(att, luo) ) AND
        citt_luo(c, luo) AND
        citt_reg(c, r) }|

```

- Use-Case Strumenti feedback

lascia_feedback(via:Viaggio, v:1..5): feedback

pre:

sia u:Utente l'istanza dell'attore che ha invocato
lo use-case

partecipa(u, via)

AND

!organizza(u, via)

AND

!feedback(u, via)

AND

EXISTS fin |

fine(via, fin) AND fin < adesso

post:

il livello estensionale finale differisce
da quello iniziale come segue:

nuovi elementi del dominio: alpha

elementi rimossi dal dominio: nessuno

nuove ennuple:

- feedback(u, via)
- voto(u, via, v)
- istante(u, via, adesso)

ennuple rimosse: nessuna

valore di ritorno: result = alpha

- Use-Case Strumenti statistiche

calcola_viaggi_città_anno(cit:Città):

(mese:1..12, nv)[12]

pre:

nessuna

post:

questo use-case non modifica il livello estensionale
result soddisfa la seguente formula:

anno_rif = anno(adesso) - 1

result { (nm, nvis) |

nm in {1,2,...,12} AND

nvis = |{ via | EXISTS att, iniv, finv, luo, ini_m, fin_m |
att_via(att, via) AND

data_inizio_mese(anno_rif, nm, ini_m) AND

data_fine_mese(anno_rif, nm, fin_m) AND

inizio(via, iniv) AND

fine(via, finv) AND

iniv <= fin_m AND

finv >= ini_m AND

```
( att_luo(att, luo) OR arrivo(att, luo) ) AND  
citt_luo(cit, luo) }| }
```